

[Edit](#)

Rebase, Stash, and Apply Commits Precisely

Challenge Overview

Understand the scenario

You are a Developer for Hexelo, an organization that needs to handle advanced Git workflows.

In this Challenge Lab, you will clean up history, save temporary changes, and selectively move commits between branches. First, you will stash uncommitted changes on a feature branch. Next, you will commit changes on a separate feature branch and merge those changes into master. Then, you will rebase master onto your original feature branch, apply your stashed changes, and resolve the merge conflict. Finally, you will drop your stashed changes and view the history.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

🔗 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Configure the Git environment and project files

Hints Enabled

No Yes

- Sign in to the virtual machine as **LabUser** by using T **Passw0rd!** as the password.
- Enter your GitHub Username and Email Address into the following associated textboxes:

GitHub Username

GitHub Email Address

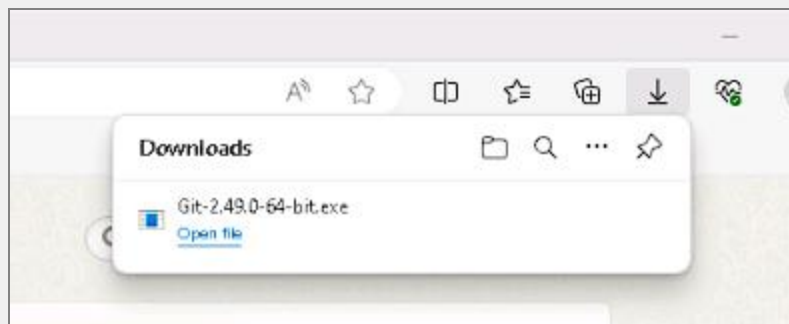
- Go to T <https://git-scm.com/download/win>, and then download and install **Git** by using the default options.

💡 Expand this hint for guidance on installing Git.

- In Microsoft Edge, open a new tab.
- In the Address bar, enter <https://git-scm.com/download/win>, and then press **Enter**.
- On the **git** Download for Windows page, in Downloads, select **Click here to download**.



- Once the download completes, select **Open file**.



- On the *User Account Control - Do you want to allow this app to make changes to your device?* dialog, select **Yes**.
- On the Git Setup page, leave *Only show new options* selected, and then select **Install**.



Git 2.49.0 Setup

Information
Please read the following important information before continuing.

When you are ready to continue with Setup, click Next.

GNU General Public License
Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330, Boston, MA 02111-1307, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change

<https://gitforwindows.org/>

☒ Only show new options

Install **Cancel**

- On the *Completing the Git Setup Wizard* page, select **Launch Git Bash**, deselect **View Release Notes**, and then select **Finish**.

 Want to learn more? Review the documentation on [installing Git](#).

- Create a project folder named `lab-git-rebase-stash` using the `mkdir` command in the **D:/LabFiles** directory and use the `cd` command to change to the new folder.

 Expand this hint for guidance on creating a project folder and changing to the new folder. 

- In the Git Bash terminal, run the following command:

```
bash
```

```
mkdir -p /d/LabFiles/lab-git-rebase-stash && cd /d/LabFiles/lab-git-reba
```

- Initialize a new Git repository by running the following command in Git Bash:

 Expand this hint for guidance on initializing a new Git repository. 

- In your Git Bash terminal, enter `T git init`, and then press **Enter** to initialize the Git repository.

 Want to learn more? Review the documentation on:

- [Initializing a new Git repository](#)
- [Git Basics](#)

- Set your Git Identity by using the `T git config` command and using `T <Name>` as the **username** and `T <GitHubEmail>` as the **email address**.

 Expand this hint for guidance on setting your Git Identity. 

- In the Git Bash Terminal, run the following commands to set your Git identity-press **Enter** on your keyboard after entering a command.

```
bash
```

```
git config --global user.name "<Name>"
```

```
bash
```

```
git config --global user.email "<GitHubEmail>"
```

 Want to learn more? Review the documentation on [setting your Git Identity](#).

 The `--global` Git identity can be changed as many times as you like (each change overwrites the previous one), and you can always override it for individual repositories using `--local`.

- Create and commit an initial file named `T sample.txt` and a base `T app.txt` file by using the `T echo`, `T git add`, and `T git commit` commands.

💡 Expand this hint for guidance on creating and committing initial files in Git. ^

- In your Git Bash terminal, run the following commands:

```
bash
```

```
echo "Line 1: master version" > sample.txt && git add sample.txt && git
```

```
bash
```

```
echo "Base app file" > app.txt && git add app.txt && git commit -m "Add
```

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (master)
$ echo "Line 1: master version" > sample.txt && git add sample.txt && git commit -m "Initial commit with sample.txt"
warning: in the working copy of 'sample.txt', LF will be replaced by CRLF the next time Git touches it
[master (root-commit) eca383c] Initial commit with sample.txt
1 file changed, 1 insertion(+)
create mode 100644 sample.txt

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (master)
$ echo "Base app file" > app.txt && git add app.txt && git commit -m "Add base app.txt"
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
[master 363c2d4] Add base app.txt
1 file changed, 1 insertion(+)
create mode 100644 app.txt

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (master)
$
```

💡 Want to learn more? Review the documentation on [creating an initial file in Git](#) and [committing a file in Git](#).

- Create a new branch named `feature/login` by using the `git switch` command.

💡 Expand this hint for guidance on creating and switching to a new feature branch. ^

- In your Git Bash terminal, enter `git switch -c feature/login`, and then press **Enter** to create and switch to a new feature branch.

💡 Want to learn more? Review the documentation on [creating and switching to a new feature branch](#).

- Modify the `app.txt` file with the text `"Login feature in progress"` by using the `echo` command.

💡 Expand this hint for guidance on modifying a file. ^

- In your Git Bash terminal, enter `T echo "Login feature in progress" >> app.txt`, and then press **Enter** to modify a file.

⚠ Do **NOT** commit. This is pending work.

Check your work

Verify

Stash uncommitted changes

Hints Enabled

No Yes

- Save your local changes by using the `T` `stash` command.

 Expand this hint for guidance on stashing your changes. 

- In your Git Bash terminal, enter `T` `git stash push -m "WIP: login in progress"`, and then press **Enter** to save your changes.

 Want to learn more? Review the documentation on the [git stash](#) command.

 You should receive a message indicating the working directory and index state were saved to the stash.

- Verify the stash is saved by using the `T` `git stash` and `T` `list` commands.

 Expand this hint for guidance on verifying the stash is saved. 

- In your Git Bash terminal, enter `T` `git stash list`, and then press **Enter**.

```
LabUser@Client01 MINGW64 ~/D:LabFileslab-git-rebase-stash (feature/login)
$ git stash push -m "WIP: login in progress"
Saved working directory and index state On feature/login: WIP: login in progress

LabUser@Client01 MINGW64 ~/D:LabFileslab-git-rebase-stash (feature/login)
$ git stash list
stash@{0}: On feature/login: WIP: login in progress

LabUser@Client01 MINGW64 ~/D:LabFileslab-git-rebase-stash (feature/login)
$ |
```

 Want to learn more? Review the documentation on [verifying the stash is saved](#).

- Switch back to the `T` `master` branch by using the `T` `git switch` command.

 Expand this hint for guidance on switching back to master. 

- In your Git Bash terminal, enter `T git switch master`, and then press **Enter**.

 Want to learn more? Review the documentation on the [git switch](#) command.

Check your work

Verify

Simulate incoming changes and rebase

Hints Enabled

No Yes

- Create a new branch named `feature/settings` to simulate a teammate's changes by using the `git switch` command.

 Expand this hint for guidance on creating a new branch to simulate changes. 

- In your Git Bash terminal, enter `git switch -c feature/settings`, and then press **Enter**.
-
- Add another change to `app.txt` by using the `echo` command, and then stage and commit the changes by using the `git add` and `git commit` commands.

💡 Expand this hint for guidance on adding changes and committing a file. ^

- In your Git Bash terminal, run the following commands:

```
bash

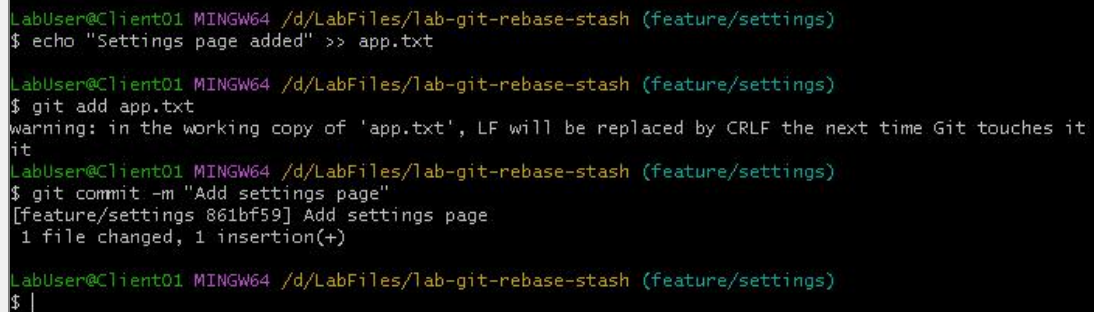
echo "Settings page added" >> app.txt

bash

git add app.txt

bash

git commit -m "Add settings page"
```



```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/settings)
$ echo "Settings page added" >> app.txt

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/settings)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/settings)
$ git commit -m "Add settings page"
[feature/settings 861bf59] Add settings page
1 file changed, 1 insertion(+)

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/settings)
$ |
```

- Switch back to master and merge the `feature/settings` branch into `master` by using the `git switch` and `git merge` commands.

💡 Expand this hint for guidance on switching back to the master and merging. ^

- In your Git Bash terminal, run the following commands:

```
bash

git switch master

bash

git merge feature/settings
```

- Switch back to the `feature/login` branch by using the `git switch` command.

💡 Expand this hint for guidance on checking out of a branch. ^

- In your Git Bash terminal, enter `T git switch feature/login`, and then press **Enter**.

- Rebase your branch onto `T master` by using the `T git rebase` command.

💡 Expand this hint for guidance on rebasing a branch. ^

- In your Git Bash terminal, enter `T git rebase master`, and then press **Enter**.

📄 *git rebase master* takes all the commits you've made on your current branch and "replays" them on top of the latest state of master.

💡 Want to learn more? Review the documentation on the [git rebase](#) command.

- Save your git bash command history by running the following command:

```
bash
history -a
```

📄 This is required for the verification scripts to run properly.

Check your work

Verify

Apply stashed changes

Hints Enabled

No Yes

- List the saved stashes again by using the `T` `git stash` command.

💡 Expand this hint for guidance on listing your stashes again. ^

- In your Git Bash terminal, enter `T` `git stash list`, and then press **Enter** to list your stashes again.

- Apply your stash (but don't drop it yet) by using the `T` `git stash` command. This will cause a merge conflict.

💡 Expand this hint for guidance on applying your stash. ^

- In your Git Bash terminal, enter `T` `git stash apply`, and then press **Enter** to apply your stash.

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/login)
$ git stash apply
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
On branch feature/login
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   app.txt

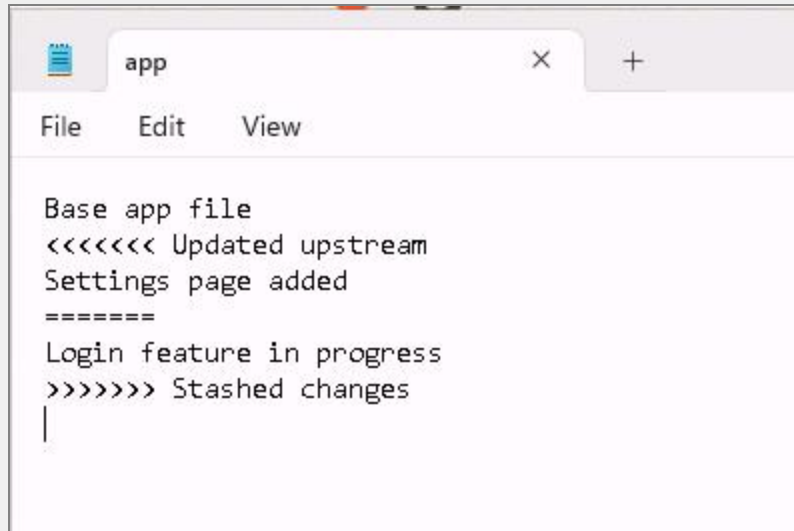
no changes added to commit (use "git add" and/or "git commit -a")
```

💡 Want to learn more? Review the documentation on [applying your stash](#).

- Remove the conflict markers in the `T` `app.txt` caused by applying the Git stash.

💡 Expand this hint for guidance on removing conflict markers in a text file. ^

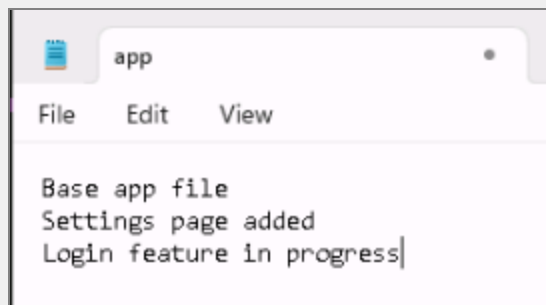
- In the Git Bash terminal, confirm the conflicted file by running `T git status`.
- Open the conflicted file by running `T notepad app.txt`.
- Locate the conflict markers (<<<<<<, =====, >>>>>>) in the file.



A screenshot of a Notepad window titled 'app'. The window has a menu bar with 'File', 'Edit', and 'View'. The text content is as follows:

```
Base app file
<<<<<< Updated upstream
Settings page added
=====
Login feature in progress
>>>>>> Stashed changes
|
```

- Edit the file to keep the desired content and remove all conflict markers.



A screenshot of the same Notepad window titled 'app'. The conflict markers have been removed, leaving only the desired content:

```
Base app file
Settings page added
Login feature in progress|
```

- **Save** and close the file.

- Confirm the changes in the `T appt.txt` file by using the `T cat` command.

💡 Expand this hint for guidance on confirming changes in a file. ^

- In your Git Bash terminal, enter `T cat app.txt`, and then press **Enter** to check the file.

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (feature/login)
$ cat app.txt
Base app file
Settings page added
Login feature in progress
```

- Stage and commit your changes by using the `T git add` and `T git commit` commands with the message `T "Continue login feature"`.

💡 Expand this hint for guidance on committing applied changes. ^

- In your Git Bash terminal, run the following commands:

```
bash
```

```
git add app.txt
```

```
bash
```

```
git commit -m "Continue login feature"
```

💡 Want to learn more? Review the documentation on the [git add](#) and [git commit](#) commands.

- Now, drop the stash if all is well by using the `T git stash drop` command.

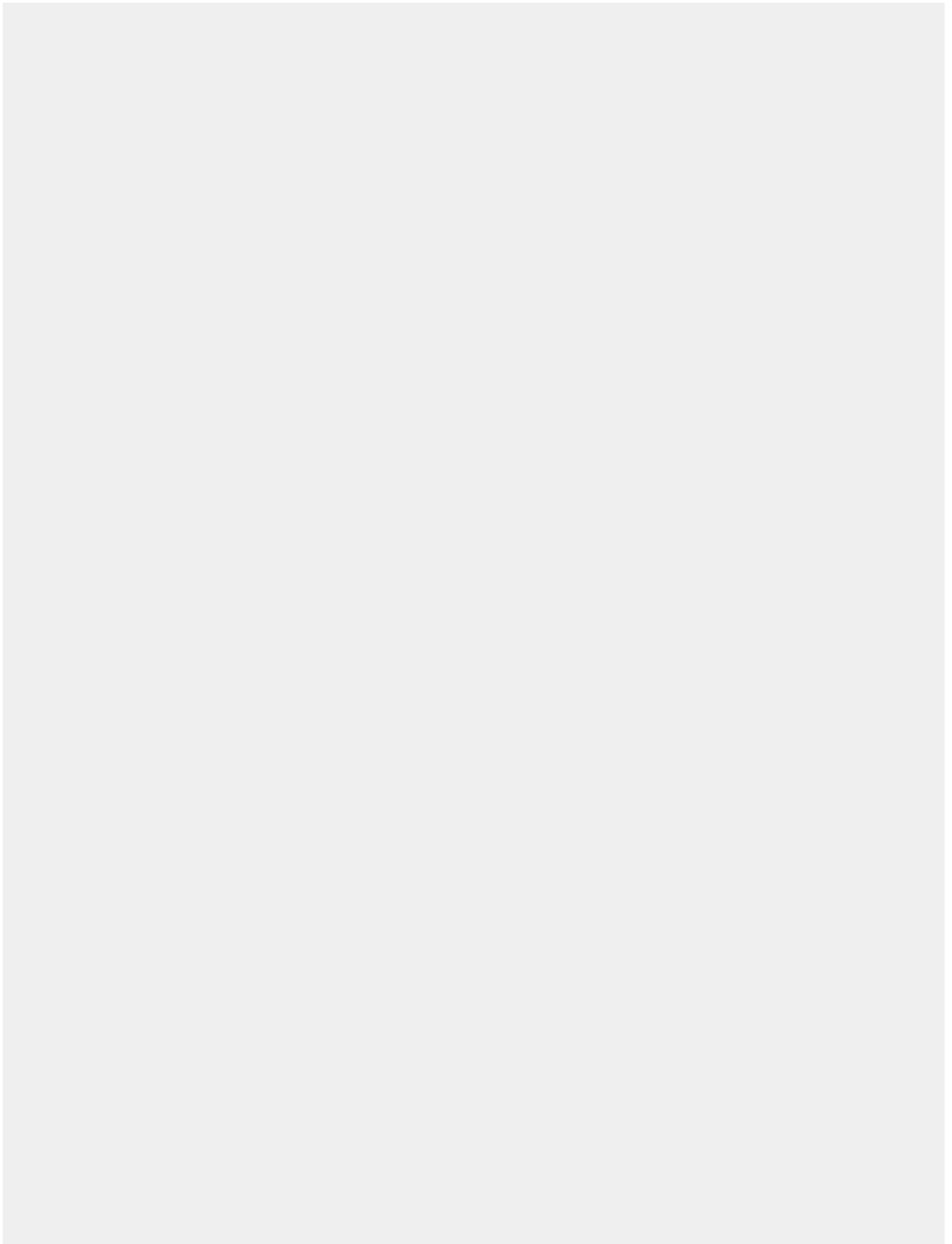
💡 Expand this hint for guidance on dropping a stash. ^

- In your Git Bash terminal, enter `T git stash drop`, and then press **Enter** to drop the stash.

💡 Want to learn more? Review the documentation on [dropping a stash](#).

Check your work

Verify



Finalize and view history

Hints Enabled

No Yes

- Switch back to `T` `master` by using the `T` `git switch` command.

 Expand this hint for guidance on switching branches. 

- In your Git Bash terminal, enter `T` `git switch master`, and then press **Enter** to switch branches.

 Want to learn more? Review the documentation on [switching branches](#).

- Merge the `T` `feature/login` branch by using the `T` `git merge` command.

 Expand this hint for guidance on merging the feature/login branch to master. 

- In your Git Bash terminal, enter `T` `git merge feature/login`, and then press **Enter** to merge branches.

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (master)
$ git merge feature/login
Updating 861bf59..e7a0a8c
Fast-forward
 app.txt | 4 ++++
 1 file changed, 4 insertions(+)

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-rebase-stash (master)
$
```

 Want to learn more? Review the documentation on [merging branches](#).

- View the commit history by using the `T` `git log` command.

 Expand this hint for guidance on viewing the commit history. 

- In your Git Bash terminal, enter `T git log --online --graph --all`, and then press **Enter** to view the commit history.

 Want to learn more? Review the documentation on [viewing the commit history](#).

Check your work

Verify

Summary

Congratulations, you have completed the **Rebase, Stash, and Apply Commits Precisely** Challenge Lab.

You have accomplished the following:

- Configured the Git environment and project files.
- Stashed uncommitted changes.
- Simulated incoming changes and rebased.
- Applied stashed changes.
- Finalized and viewed history.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Track Local Changes and Manage Commits [Guided]
- Create Branches and Merge Changes Safely [Guided]
- Collaborate Remotely and Use Pull Requests [Guided]
- Build a Real Project by Using Git Workflows [Advanced]

- Write Code with GitHub Copilot Suggestions [Guided]
- Control Git Behavior with Config and Ignore Files [Guided]
- Rebase, Stash, and Apply Commits Precisely [Guided]
- Manage Tasks by Using GitHub Issues and Boards [Guided]
- Automate Testing and CI with GitHub Actions [Guided]
- Review and Refactor Code with Copilot [Guided]
- Implement Release Management and Versioning in GitHub [Expert]

